Machine Learning Practical Application 1-2
Avalanche Risk Prediction

Alexia Miriam Muresan Pop

November 5, 2026

1 Introduction

Avalanche forecasts provide information about the expected level of avalanche danger in a certain region and period. This information is particularly important for mountain activities and winter sports, where understanding the expected snow conditions can help people make safer decisions.

The aim of this assignment is to investigate whether supervised machine learning algorithms can learn to classify avalanche forecasts according to their expected danger level, using weather conditions, geographical information, and other variables available in the forecast data.

Avalanche danger can arise from different types of snow instability, which are described by the European Avalanche Warning Services [1]:

- **New snow:** Recent snowfall adds weight to the existing snowpack, potentially creating unstable layers.
- **Wind slab:** Wind transports and deposits snow, forming dense accumulations that can be triggered on certain slopes.
- **Persistent weak layers:** Weak layers within the existing snowpack can remain unstable for weeks or months, sometimes allowing avalanches to be triggered remotely.
- **Wet snow:** Melting or rainfall introduces liquid water into the snowpack, weakening its structure and potentially causing wet avalanches.
- **Gliding snow:** The entire snowpack slides over the ground, usually due to reduced friction. These avalanches are particularly difficult to predict.

Among these, the variables available in this dataset relate mainly to new snow (precipitation), wind slab (wind speed and direction) and wet snow (rainfall and temperature). Persistent weak layers and gliding snow depend on the history and structure of the snowpack, which is not included in the data, so they cannot be captured directly by the models.

It is important to note that the models trained in this assignment do not estimate the probability that an avalanche will occur, but rather they try to predict the danger categories assigned to forecasts by the Norwegian Water Resources and Energy Directorate (NVE) based on the information available in the dataset.

2 Dataset

2.1 Data source and dataset creation

The data was collected from the NVE avalanche forecast API, with documentation found in [3].

The original warnings are given in JSON format. Each warning contains information about the region, weather conditions, expected danger level, possible consequences of the avalanche and even text descriptions for advice.

In particular, I constructed the dataset by extracting all the warnings that were issued from 1st December to 31st May (official forecasting period) from the years 2023 to 2026.

The JSON file was processed using Python to obtain a structured dataset in CSV format, with each row representing a forecast. Not all the variables were extracted for the CSV, only the ones accounting for the preexisting conditions in order to only take into account information before the forecast was issued. This process is documented in more detail in my GitHub repository.

2.2 Basic description

- The dataset contains 13190 observations with 22 predictor variables and the target variable.
- The target variable, `risk` represents the expected danger of the avalanche and it has three categories: Low (3539 observations), Medium (6915 observations) and High (2736 observations), with Medium being the most prevalent class.

2.3 Predictor variables

Variable	Type	Description
Geographical variables		
`RegionName`	Categorical	Name of the avalanche forecasting region.
`geographical_region`	Categorical	Broader geographical region: Arctic, Northern Norway, Trøndelag, Western Norway...
Forecast timing variables		
`publish_month`	Integer	Month in which the forecast was published.
`publish_hour`	Integer	Hour of the day at which the forecast was published.
Weather variables		
`cloud_cover`	Categorical	Type of cloud conditions: cloudy, storm, clear skies...
`rainfall_most_exposed_area`	Numeric	Rainfall in the most exposed area of the mountain (mm).
`rainfall_average`	Numeric	Average rainfall (mm).
`wind_speed`	Categorical	Prevailing wind speed: breeze, gale, no wind...
`wind_direction`	Categorical	Prevailing wind direction: N, NE, S, W...
`wind_shift_speed`	Categorical	Expected wind speed change. It uses the same categories as `wind_speed`.
`wind_shift_direction`	Categorical	Expected wind direction change. It uses the same categories as `wind_direction`.
`wind_direction_change`	Numeric	Wind direction change (degrees).
`temperature_min`	Numeric	Minimum temperature (ºC).
`temperature_max`	Numeric	Maximum temperature (ºC).

Variable	Type	Description
`freezing_level_masl`	Numeric	Altitude of the freezing level, expressed in meters above sea level (masl).
`rain_snow_line_altitude_masl`	Numeric	Altitude at which falling snow turns to rain (masl).
Weather-related temporal variables		
`wind_shift_start`	Integer	Expected start hour of the forecasted wind shift.
`wind_shift_end`	Integer	Expected end hour of the forecasted wind shift.
`freezing_level_start`	Integer	Expected start hour of the forecasted freezing level.
`freezing_level_end`	Integer	Expected end hour of the forecasted freezing level.
`rain_snow_line_start`	Integer	Expected start hour of the forecasted rain-snow line.
`rain_snow_line_end`	Integer	Expected end hour of the forecasted rain-snow line.

2.4 Data preprocessing

2.4.1 Derived features

- Extract `publish_month` and `publish_hour` from the original `Publish_Time` which was in format YYYY-MM-DDTHH:mm:ss.SS in order to have meaningful numerical features.
- Derive `wind_direction_change` from `wind_direction` and `wind_shift_direction` to obtain a numerical expression for the degrees of wind direction change. This was done using compass directions. For example a change from N to S would be 180°.
- Derive `geographical_region` from `RegionName` in order to obtain a broader geographical region. Since specific slope and geographic datasets were unavailable, this broader categorization could help the models by providing intrinsic geographic characteristics.
- The target variable `risk` was created by reclassifying `DangerLevel` [2] into three categories: Low (Level 1, small, $n = 3539$), Medium (Level 2, moderate, $n = 6915$), and High (Levels 3 and 4, significant and big, $n = 2503$ and $n = 233$, respectively). Level 0 observations were excluded as they contained no danger information, and Level 5 (very big) too because there were no observations. To address the severe low count in Level 4, the Levels 3 and 4 were merged into High based on their shared high-risk connotation, resulting in a more balanced distribution.

2.4.2 Null values

The way of dealing with null values was motivated by the text description of the weather measurements in the JSON file. From there I could draw the conclusion that missing values meant that said phenomenon was not expected to happen or in the case of start-time variables, that the phenomenon was already happening before the warning.

- If a start weather-related variable didn't have a value, it was replaced by 0 because there was no expected starting time.
- Missing values in `wind_speed` or `wind_direction` were replaced by "No wind".
- Missing values in `wind_shift_direction` and `wind_shift_speed` and were replaced with the prevailing values as they meant that there was no wind direction alteration or no wind at all respectively.
- Missing values in `cloud_cover` were replaced by "Not given" since it was an already existing category.

2.4.3 Cyclic Encoding

Cyclic features like hour and month have temporal patterns that a numerical scale doesn't capture. For example, month 12 is right next to month 1, not 12 times larger. To fix this, these features were mapped using the following transformations:

$$x_{\sin} = \sin\left(\frac{2\pi x}{T}\right), \quad x_{\cos} = \cos\left(\frac{2\pi x}{T}\right)$$

where x is the value and T is the period (24 for hours, 12 for months).

This transformation was applied to all temporal and forecast timing variables before training the k-NN and Logistic Regression models.

3 Exploratory Analysis

3.1 Risk Distribution

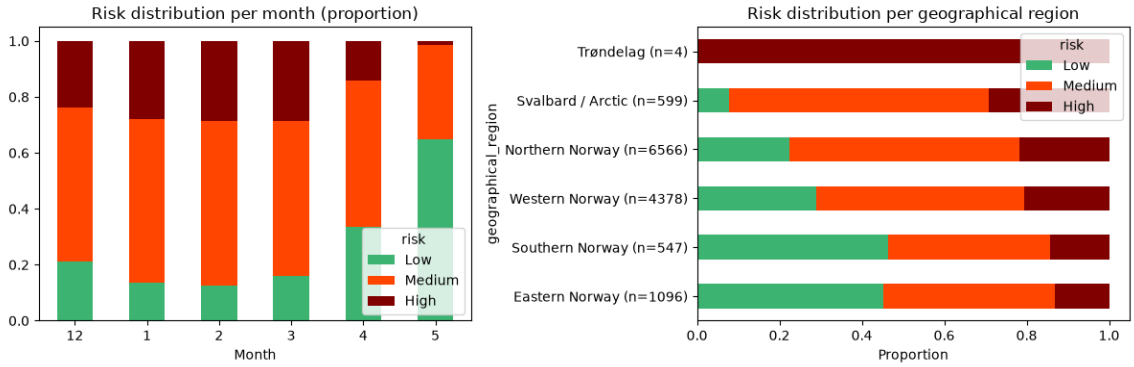


Figure 1: Risk distribution across months and geographical regions

In Figure 1, High and Medium danger forecasts appear more frequently during the winter months, whereas Low danger forecasts are relatively more common towards the end of winter and in early spring.

The geographical distribution also shows differences across Norway. High and Medium danger forecasts are more frequent in the northern and western regions, while Low danger forecasts account for a larger proportion of warnings in some southern and eastern regions.

These patterns suggest that avalanche danger levels are associated with both seasonal and geographical factors. However, the distributions alone do not establish which environmental conditions explain these differences.

3.2 Correlation between numerical features

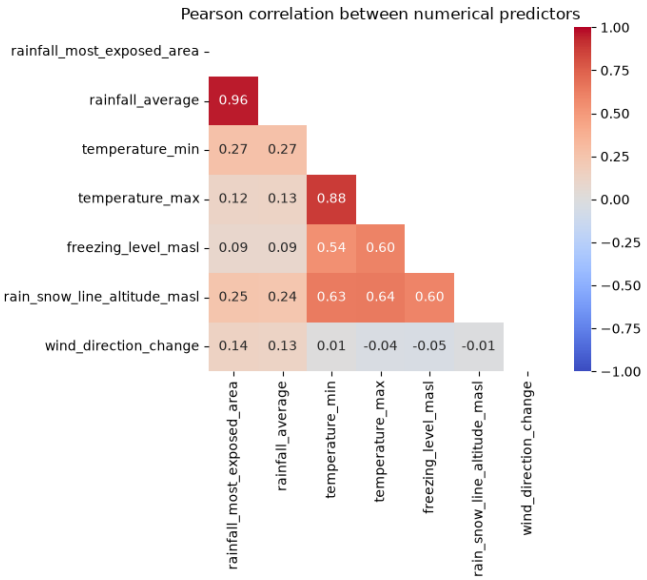


Figure 2: Correlation heatmap for numerical features

Figure 2 shows the Pearson correlation matrix between the numerical predictors (excluding temporal variables). The strongest correlation is observed between `rainfall_most_exposed_area` and `rainfall_average` ($r = 0.96$). This is physically reasonable, since periods with higher average rainfall will generally also experience higher rainfall in the most exposed areas. Their strong correlation suggests that including both variables may provide limited additional information.

Minimum and maximum temperature also have a strong correlation ($r = 0.88$), which is expected because both variables reflect the general thermal conditions of a region. Furthermore, temperature is positively correlated with the freezing level ($r = 0.54$ and 0.60) and the altitude of the rain-snow line ($r = 0.63$ and 0.64). These relationships are physically meaningful: warmer atmospheric conditions means that freezing and the conversion of snow to rain happens at a higher altitude.

Correlated features may introduce unstable coefficients in Logistic Regression or redundancy in k-NN. However, these variables may still capture distinct aspects of the meteorological conditions. Therefore, all variables were retained in the modeling stage, and the correlations are taken into account when interpreting the results.

4 Methodology and evaluation

Several classification algorithms were implemented and compared: k-Nearest Neighbours, C4.5, RIPPER, Support Vector Machine, Logistic Regression, Stacking, AdaBoost and Random Forest. For each algorithm, different parameters were explored when it was possible. Additionally, three feature configurations were compared: all variables, univariate feature selection and multivariate feature selection. Wrapper-based feature selection was also evaluated for the algorithms where it was computationally viable.

Although overall accuracy is important, one of the main priorities is to minimize High and Medium danger instances misclassified as Low, since these errors could have serious real-life consequences. In contrast, misclassifying Low as Medium or High mainly results in false alarms. Misclassifying High as Medium is also undesirable, although a Medium warning may still en-

courage sufficient precautionary measures.

Therefore, particular attention was paid to the *High* → *Low* and *Medium* → *Low* misclassification rates, as well as class-specific recall. These criteria were considered when comparing and selecting the final models.

Finally, each subsection focuses on a specific classification algorithm, presenting and discussing its performance and interpreting the results obtained. Once all algorithms have been analyzed, their advantages and disadvantages are compared, considering both overall performance and their ability to identify the different danger levels. This final comparison provides the basis for selecting the most suitable model for the classification task.

5 Non-Probabilistic Algorithms

5.1 k-Nearest Neighbors (IBk)

5.1.1 Selection of the Number of Neighbors k

To select a suitable value for k, I analyzed the performance of the model using all variables for odd values of k between 1 and 31 with Python.

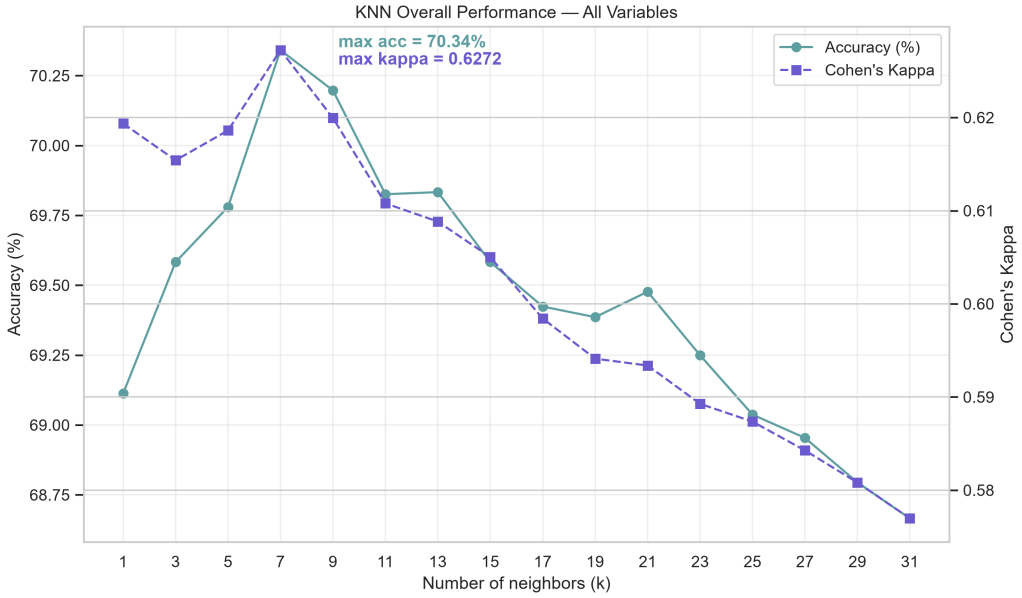


Figure 3: Accuracy and Quadratic Weighted Kappa for different values of k.

Figure 3 shows the evolution of accuracy and Quadratic Weighted Kappa as k increases. I used quadratic weight in order to penalize extreme errors like High/Medium to Low. However, these metrics are not enough to select the optimal value of k, since underestimating the danger level as Low is an important error. Therefore, the recall of each class and the corresponding underestimation rates were also examined.

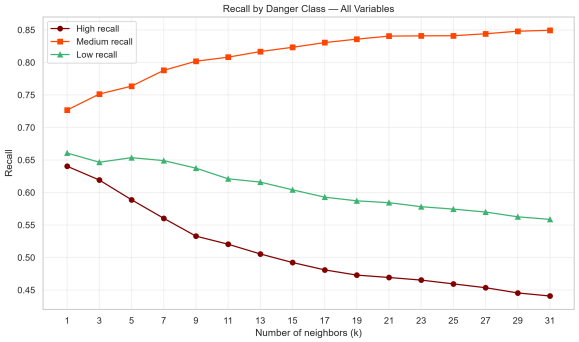


Figure 4: Recall for each danger class for different values of k

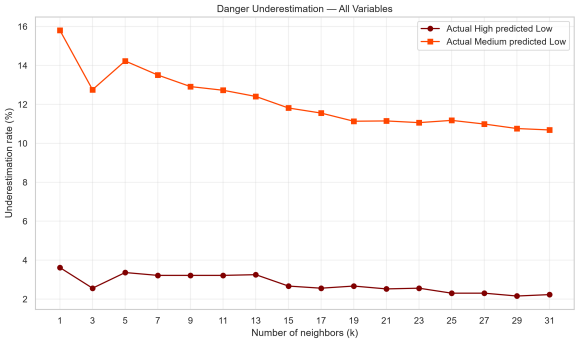


Figure 5: Underestimation rates for different values of k

The previous figures illustrate the trade-off between the three classes. As k increases, recall for the Medium class generally improves, whereas recall for the High class decreases as seen in Figure 4. One possible explanation, supported by Figure 5, is that as the number of neighbors increases, the model is more likely to classify instances as Medium, since it is the predominant class. As a result, some instances that would otherwise be incorrectly classified as Low are correctly classified as Medium. However, this comes at the expense of more High instances being misclassified as Medium. This suggests that the High and Medium classes share similar characteristics, making them difficult to distinguish.

Consequently, selecting k solely on the basis of overall accuracy would not necessarily provide the most appropriate balance. Considering safety as an important factor, $k = 3$ was selected as a compromise since accuracy only drops 0.75% and High/Medium to Low misclassification remains relatively low.

Therefore, the Univariate, Multivariate and Wrapper analyses will be carried out using $k = 3$.

5.1.2 Model Interpretation

Metric	All variables	Univariate	Multivariate	Wrapper
Accuracy (%)	**69.58**	63.77	63.84	67.92
Cohen's Kappa	**0.496**	0.402	0.397	0.472
Quadratic Kappa	**0.615**	0.531	0.533	0.598
Precision	**0.695**	0.637	0.637	0.679
Recall	**0.696**	0.638	0.638	0.679
F1-score	**0.695**	0.637	0.636	0.679
AUC	**0.801**	0.752	0.746	0.783
RMSE	**0.384**	0.417	0.418	0.403
Recall (High)	**0.6192**	0.5917	0.5870	0.6093
Recall (Medium)	**0.7514**	0.6954	0.7121	0.7210
Recall (Low)	0.6465	0.5603	0.5343	**0.6516**
High → Low (%)	**2.56**	3.73	3.33	3.73
Medium → Low (%)	**12.75**	16.33	15.63	15.72
Low → High (%)	4.15	6.24	5.23	**3.84**

Table 2: k-NN performance for $k = 3$

Table 2 shows that the All Variables configuration achieves the best overall performance and the lowest rates of misclassifying High and Medium observations as Low.

These results suggest that the variables removed during Univariate and Multivariate feature

selection contain useful information for distinguishing between danger levels. By retaining all variables, k-NN can consider a wider range of characteristics, helping it distinguish High and Medium classes from Low danger.

However, this comes at the expense of distinguishing between High and Medium observations, as reflected in the confusion matrix, where a considerable number of High observations are misclassified as Medium. This suggests that, although using all variables improves the identification of danger levels relative to Low, the overlap in characteristics between High and Medium observations remains a challenge.

	Predicted class		
Actual class	High	Medium	Low
High	1694	972	70
Medium	837	5196	882
Low	147	1104	2288

Table 3: Confusion matrix for All Variables setting

5.2 Classification Tree C4.5 (J48)

Metric	**All variables**	**Univariate**	**Multivariate**	**Wrapper**
Accuracy (%)	**66.55**	65.67	66.10	66.31
Cohen's Kappa	**0.4126**	0.3986	0.4098	0.4099
Precision	**0.673**	0.663	0.666	0.670
Recall	**0.666**	0.657	0.661	0.663
F1-score	**0.655**	0.647	0.652	0.653
AUC	**0.749**	0.738	0.747	**0.749**
RMSE	0.3989	0.4023	0.3999	**0.3986**
Recall (High)	**0.493**	0.485	0.487	0.481
Recall (Medium)	**0.825**	0.810	0.802	0.815
Recall (Low)	0.486	0.490	**0.519**	0.507
High → Low (%)	**1.61**	1.79	2.01	2.27
Medium → Low (%)	**10.24**	12.08	12.23	11.64
Low → High (%)	1.47	1.07	1.27	**0.68**

Table 4: J48 performance across feature selection

The results show that feature selection does not improve J48. In fact all metrics in the first table remain almost identical across settings, excluding the misclassification errors in which the All Variables setting dominates.

This is likely due to how J48 handles features. Because decision trees select only informative attributes at each split, unhelpful features are automatically excluded. Consequently, feature selection offered no performance improvement in this case.

The tree was configured with a confidence factor of 0.01, a minimum of 100 instances per leaf, and binary splits enabled in order to have a smaller tree with more interpretability and representative splits.

Overall, the model using all variables is the strongest choice based on both the metrics and the critical misclassification errors.

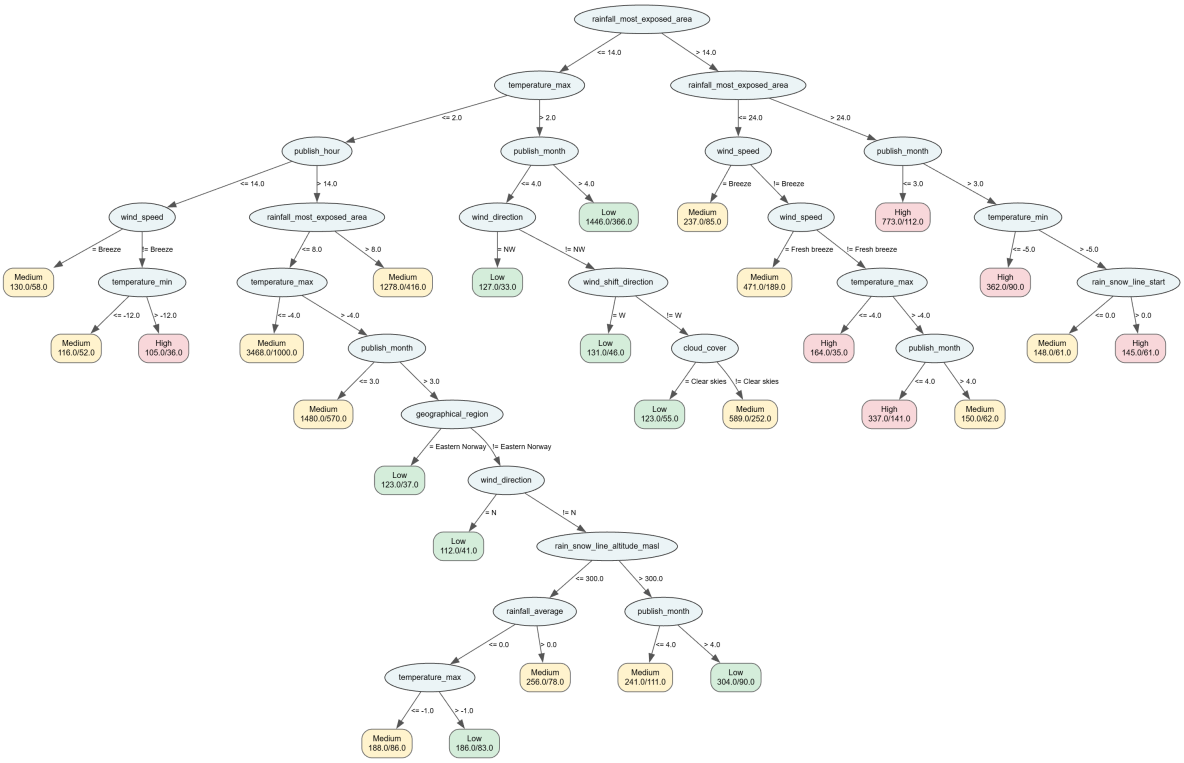


Figure 6: Decision tree obtained using J48 and all variables.

The resulting tree seems to be consistent with real life phenomena. `rainfall_most_exposed_area` seems to be the deciding factor, which makes sense because high precipitation can cause wet-snow avalanches.

High danger is predicted mainly in wet winter forecasts as we can see from the following rule .

> `rainfall_most_exposed_area > 24 AND publish_month <= 3` $\Rightarrow$ `High (773/112)`

It classifies 773 forecasts as High, of which 661 (85.5%) are correct. The same rule was found independently by JRip (Section 5.3.2), which supports its reliability.

Another example is the rule

> `rainfall_most_exposed_area <= 14 AND temperature_max > 2`
> `AND publish_month > 4` (May) $\Rightarrow$ Low (1446/366)

It classifies 1446 observations as Low, of which 366 (25.3%) are actually Medium or High. This result is intuitive, as low precipitation and warmer temperatures during spring are generally associated with less favorable conditions for avalanche formation. Still, a quarter of its predictions are underestimations, which is the error type that matters most in this project.

However, it must be stated that some branches might be overfitted. For example the rule

> `rainfall_most_exposed_area <= 14, AND temperature_max > 2 AND`
> `publish_month <= 4 AND wind_direction = NW` $\Rightarrow$ Low (127/33)

classifying as Low. The wind direction doesn't really explain anything without a magnitude associated or a reference system, so the model also learns meaningless patterns in some cases.

Note: The tree was rendered by Weka, but I used the library graphviz in Python to obtain a more visually aesthetic tree.

5.3 Rule Induction RIPPER (JRip)

5.3.1 Selection of the minimum number of instances per rule `minNo`

The JRip classifier was evaluated using 10-fold cross-validation for five values of the minimum number of instances per rule, `minNo` $\in \{2, 10, 20, 30, 50\}$.

Metric	2	10	20	30	50
Accuracy (%)	68.52	**68.63**	68.31	67.38	66.66
Cohen's Kappa	0.4522	**0.4530**	0.4485	0.4322	0.4223
Precision	0.693	**0.695**	0.690	0.681	0.671
Recall	0.685	**0.686**	0.683	0.674	0.667
AUC	**0.745**	0.743	0.742	0.733	0.729
RMSE	0.3924	**0.3919**	0.3934	0.3960	0.3978
Recall (High)	0.487	0.490	**0.501**	0.470	0.484
Recall (Medium)	0.821	**0.824**	0.819	0.809	0.796
Recall (Low)	**0.574**	0.568	0.558	0.568	0.555
Number of Rules	75	59	36	**23**	20

Table 5: JRip performance across different values of `minNo` with all variables

Increasing `minNo` generally reduced the number of rules, resulting in a simpler and more interpretable model, although predictive performance tended to slightly decrease.

Although `minNo` = 10 achieved the highest overall accuracy (68.63%), `minNo` = 20 provided the highest recall for the High-risk class (0.501) while retaining almost the same accuracy and reducing the number of rules from 59 to 36. Given the importance of identifying high and medium avalanche danger, `minNo` = 20 was selected as a compromise between accuracy, High-risk recall and number of rules.

This value was therefore kept fixed in the analysis for Univariate, Multivariate, and Wrapper feature selection.

5.3.2 Model Interpretation

Metric	All variables	Univariate	Multivariate	Wrapper
Accuracy (%)	**68.31**	67.10	67.43	66.73
Cohen's Kappa	**0.4485**	0.4315	0.4329	0.4206
Precision	**0.690**	0.676	0.681	0.673
Recall	**0.683**	0.671	0.674	0.667
F1-score	**0.675**	0.664	0.666	0.659
AUC	**0.742**	0.733	0.732	0.728
RMSE	**0.3934**	0.3975	0.3959	0.3978
Recall (High)	**0.501**	0.497	0.487	0.492
Recall (Medium)	**0.819**	0.796	0.812	0.808
Recall (Low)	0.558	**0.562**	0.551	0.529
High $\rightarrow$ Low (%)	1.72	2.59	1.75	**1.57**
Medium $\rightarrow$ Low (%)	**11.27**	13.34	12.16	11.80
Low $\rightarrow$ High (%)	**0.40**	0.62	0.54	0.88
Number of Rules	36	22	27	30

Table 6: JRip performance across feature selection methods (`minNo` = 20)

The model with all variables was selected as the final model because it provides a good trade-off between predictive performance and interpretability. Although it has the highest amount of rules, this comes with the benefit of having the least amount of Medium to Low misclassification errors and the second least amount of High to Low misclassifications (1.72% vs 1.57%).

For High-risk predictions, several rules emphasize high precipitation, particularly when combined with low temperatures of winter months or high wind speed. For example:

`(rainfall_most_exposed_area` $\geq$ `25) AND (publish_month` $\leq$ `3)` $\Rightarrow$ `High` (773/112)

This rule covers 773 instances, of which 112 are incorrectly classified. Or:

`(rainfall_most_exposed_area` $\geq$ `15) AND (publish_month` $\leq$ `4) AND`
`(wind_speed = Moderate gale)` $\Rightarrow$ `High` (133.0/39.0)

In contrast, the Low-risk rules often combine warmer temperatures with lower precipitation associated with springtime. For example:

`(temperature_max` $\geq$ `0) AND (publish_month` $\geq$ `5) AND (rainfall_average` $\leq$ `5)`
`AND (wind_speed = Breeze) AND (rainfall_most_exposed_area` $\geq$ `1)`
$\Rightarrow$ `Low` (288.0/84.0)

The model's default rule predicts Medium risk when none of the previous 35 rules apply (8417.0/2648.0). This reflects the difficulty of distinguishing the Medium danger instances from High and Low, as well as the predominance of Medium-risk observations in the dataset.

5.4 Support Vector Machine (SMO)

	Linear Kernel (C = 1.0)			**RBF Kernel ($\gamma = 0.01$)**		
Metric	**All**	**Univariate**	**Multivariate**	**All**	**Univariate**	**Multivariate**
Accuracy (%)	**68.99**	67.01	67.24	67.11	64.15	64.44
Cohen's Kappa	**0.4534**	0.4099	0.4146	0.4038	0.3446	0.3525
Precision	**0.705**	0.689	0.691	0.699	0.668	0.669
Recall	**0.690**	0.670	0.672	0.671	0.641	0.644
F1-score	**0.680**	0.655	0.658	0.652	0.617	0.622
AUC	**0.757**	0.735	0.737	0.734	0.704	0.707
RMSE	**0.3909**	0.3998	0.3984	0.3993	0.4117	0.4105
Recall (High)	**0.485**	0.441	0.453	0.384	0.324	0.340
Recall (Medium)	0.849	**0.859**	**0.859**	0.883	0.870	0.865
Recall (Low)	**0.537**	0.478	0.476	0.480	0.441	0.449
High $\rightarrow$ Low (%)	1.06	1.28	1.28	**0.88**	1.02	0.91
Medium $\rightarrow$ Low (%)	10.54	10.05	9.78	**8.37**	9.40	9.92
Low $\rightarrow$ High (%)	0.51	0.76	0.51	**0.48**	0.59	0.62

Table 7: SVM performance across kernels and feature selection configurations

The linear kernel achieved the best overall results in general metrics and class-specific recall, particularly when using all variables. This suggests that the relationships captured by a linear decision boundary are enough to identify some of the patterns in the dataset, without requiring a more complex non-linear kernel.

In contrast, the RBF kernel made less errors in High/Medium to Low misclassifications. However, this came at the expense of High and Low danger detection in their recall scores.

The difference between kernels may be related to the complexity of the decision boundaries they learn. The linear kernel assumes that classes can be separated through linear boundaries in the feature space, whereas the RBF kernel can capture more complex, nonlinear relationships, which don't generalize well in this specific dataset. It could be the case that RBF performance improves with hyperparameter tuning.

Regarding feature selection, using all variables produced the best overall results for both kernels. This suggests that the removed predictors contain complementary information that helps the SVM distinguish between danger levels. Wrapper selection was not performed for SVM due to its computational cost: after 5 hours, no results were given.

Overall, the linear SVM using all variables was the strongest configuration. Although the RBF kernel reduced some direct misclassifications between High and Low danger, its considerably lower High-class recall makes it less useful for this classification task. Nevertheless, both kernels show difficulties distinguishing adjacent danger levels.

To interpret the linear SVM, we can examine the attribute weights of its three pairwise classifiers:

- Rainfall clearly dominates: `rainfall_most_exposed_area` (-5.21, -8.22 and -5.65 for High/Medium, High/Low and Medium/Low) and `rainfall_average` (-4.77, -5.34, -2.90) have by far the largest weights, and their sign favors the more dangerous class in each pair, which again is consistent with wet-snow avalanche behavior. However, the two rainfall variables are strongly correlated ($r = 0.96$), so their weights should be interpreted jointly
- Maximum temperature and freezing level have the opposite sign in each pair and push towards the less dangerous class (up to 2.04 for `temperature_max` in Medium/Low and 1.96 for `freezing_level_masl` in High/Low). This seems to imply that higher temperature (and therefore higher freezing level too) seems more relevant for Low risk.
- No wind speed category exceeds 1.1 in absolute value, and `wind_direction_change` is close to zero, so the SVM relies much more on precipitation, temperature and season than on wind.
- Regional weights agree with the logistic regression odds ratios intuition (Section 6.1): northern regions Nord-Troms (-1.04 for High/Low) lean towards higher danger, while southern regions like Heiane (0.7873 for Medium/Low) lean towards lower danger. However, most regions have moderate and low weights in general so they are not so informative.

6 Probabilistic Algorithms

6.1 Logistic Regression (Logistic)

Metric	All Variables	Univariate	Multivariate	Wrapper
Accuracy (%)	69.17	67.24	67.66	**69.64**
Cohen's Kappa	0.4657	0.4240	0.4313	**0.4708**
Precision	0.699	0.682	0.687	**0.706**
Recall	0.692	0.672	0.677	**0.696**
F1-score	0.685	0.662	0.666	**0.689**
AUC	**0.812**	0.789	0.790	0.808
RMSE	**0.3739**	0.3860	0.3850	0.3758
Recall (High)	**0.533**	0.498	0.513	0.524
Recall (Medium)	0.820	0.832	**0.836**	0.835
Recall (Low)	**0.564**	0.495	0.493	0.558
High → Low (%)	1.39	1.79	**1.09**	1.54
Medium → Low (%)	11.67	10.68	**10.20**	10.78
Low → High (%)	0.65	0.94	0.88	**0.62**

Table 8: Logistic performance across feature selection

The four Logistic Regression configurations achieve relatively similar overall performance, both in terms of evaluation metrics and misclassification rates.

However, the Wrapper approach achieves misclassification rates close to those of the Multivariate configuration, while maintaining a Low recall close to that of the All Variables configuration. It also achieves the highest Cohen's Kappa, indicating the strongest agreement beyond chance among the four configurations.

Therefore, despite the relatively small differences in overall accuracy, the Wrapper model offers the best balance between overall predictive performance and misclassification errors.

In the following table we can see some selected odds ratios for the Wrapper model against the reference class Low.

Variable	High	Medium
Region: Svalbard øst	1.83E13	83.4531
Region: Nordenskiöld Land	5.3198	3.6836
Region: Finnmarkskysten	0.1962	0.3893
Region: Nord-Troms	4.6144	3.0552
Region: Lyngen	6.3034	3.4228
Region: Indre Troms	4.4732	3.5457
Region: Jotunheimen	7.3303	3.2602
Region: Helgeland	0.3037	0.5460
Region: Hallingdal	0.4617	0.4192
Region: Hardanger	0.2664	0.5400
Region: Heiane	0.0851	0.3040
Rainfall (most exposed area)	1.1852	1.1124
Average rainfall	1.0469	0.9853
Wind speed: Storm	4.37E12	3.22E11
Wind speed: Strong gale	4.7658	1.9900
Wind speed: Gale	2.8756	1.6137
Wind speed: Moderate gale	2.2290	1.4834
Wind speed: Strong breeze	1.5261	1.2477
Wind speed: Fresh breeze	0.8457	0.8676
Wind speed: Breeze	0.4639	0.7492
Wind shift speed: Storm	4.54E11	0.0116
Wind shift speed: Moderate gale	1.8164	1.2618
Wind shift speed: Gale	2.7727	1.5536
Wind shift speed: Strong breeze	1.3275	0.9990
Wind shift speed: Fresh breeze	0.8286	1.0449
Wind shift speed: Breeze	0.6653	0.8988
Minimum temperature	0.9625	0.9664
Maximum temperature	0.9038	0.9299
Publication hour (sine)	4.6612	1.8208
Publication hour (cosine)	4.0395	3.0457
Publication month (sine)	24.1387	3.9854
Publication month (cosine)	5.2544	2.0325
Freezing level start (sine)	1.1270	1.2600

Table 9: Selected odds ratios for Logistic Regression model (Wrapper)

Both rainfall variables have odds ratios greater than 1 for High versus Low danger. For example, the odds ratio for rainfall in the most exposed area is 1.1852, indicating approximately 18.5% higher relative odds of High rather than Low danger for a one-unit increase in this variable, holding the other predictors constant. Conversely, maximum temperature has an odds ratio of 0.9038, corresponding to approximately 9.6% lower relative odds of High versus Low danger per unit increase in temperature. The two rainfall variables are strongly correlated, so their individual odds ratios should be interpreted jointly rather than separately. Overall, these relationships are consistent with the potential influence of rainfall and temperature on avalanche danger.

This analysis can also be extended to categorical variables, such as wind conditions and geographical regions. For example, the category `wind_speed = Strong gale` has an odds ratio of 4.7658 for High versus Low danger, indicating substantially higher relative odds of High rather than Low danger compared with the reference wind-speed category, all else being equal. Similarly, the region Nord-Troms has an odds ratio of 4.6144 for High versus Low danger compared with the reference region. Several northern and inland regions (Lyngen, Indre Troms, Jotunheimen) have high odds ratios, while others, such as Finnmarkskysten (0.1962), have lower odds than the reference. The regional differences therefore seem to reflect local climate and snow conditions rather than a simple north-south pattern.

Finally, some odds ratios are exceptionally large, particularly for `Region = Svalbard` and the Storm wind categories. These estimates may be unstable due to sparse observations. For example, `wind_speed = Storm` occurs in only 22 of the 13190 observations, which could help explain its extremely large odds ratio. Consequently, these odds ratios should be interpreted with caution.

6.2 Stacking (Stacking)

Stacking was implemented using JRip, C4.5 and k-NN ($k = 3$) as base classifiers using the previous parameter configurations and Logistic Regression as the meta-classifier.

Metric	All Variables	Univariate	Multivariate
Accuracy (%)	**71.24**	68.53	68.83
Cohen's Kappa	**0.5024**	0.4523	0.4595
Precision	**0.720**	0.693	0.695
Recall	**0.712**	0.685	0.688
F1-score	**0.707**	0.678	0.682
AUC	**0.827**	0.798	0.800
RMSE	**0.3653**	0.3812	0.3795
Recall (High)	**0.565**	0.523	0.533
Recall (Medium)	**0.836**	0.822	0.818
Recall (Low)	**0.585**	0.543	0.554
High → Low (%)	**1.21**	1.54	1.39
Medium → Low (%)	**9.95**	11.24	11.63
Low → High (%)	**0.37**	0.51	0.71

Table 10: Stacking performance across feature selection

Table 10 shows that Stacking with all variables achieves the best overall performance in all the metrics including the misclassification errors.

The improvement obtained with all variables suggests that the removed predictors contain useful information for the ensemble. Different base classifiers may exploit different characteristics of the data: for example, k-NN relies on distances between observations, C4.5 builds a tree of attribute splits, and JRip identifies decision rules. As a consequence, keeping more variables allows the base classifiers to identify different relationships that help each classifier.

Variable	High	Medium
JRip: High	24.4353	8.4453
JRip: Medium	0.5939	0.8502
JRip: Low	0.1044	0.1926
C4.5: High	15.4845	2.6690
C4.5: Medium	1.8000	1.5138
C4.5: Low	0.0579	0.3096
k-NN: High	22.7048	3.0361
k-NN: Medium	1.0654	1.9346
k-NN: Low	0.0622	0.1858

Table 11: Odds ratios of Logistic Regression meta-classifier

Table 11 provides insight into how Stacking combines the predictions of the three base models to determine the final danger level.

Predictions of High danger are the strongest indicators of a final High classification across all three base models. JRip and k-NN show the strongest associations, multiplying the odds of predicting High rather than Low by approximately 24.44 and 22.70, respectively, while C4.5 yields an odds ratio of 15.48. This suggests that the meta-classifier places considerable weight on High predictions, particularly those from JRip and k-NN, while also incorporating evidence from C4.5.

Predictions of Low danger consistently support the final Low classification. For example, a Low prediction from k-NN reduces the relative odds of a final High rather than Low classification by approximately 93.8% (odds ratio of 0.0622) with similar metrics for C4.5 and JRip. This consistent pattern indicates that the meta-classifier treats Low predictions as strong evidence against a High classification. The same tendency is observed for Medium versus Low, although the reduction is less so.

Predictions of Medium danger are more ambiguous with JRip reducing the odds of a final High/Medium rather than Low, while C4.5 and k-NN do the exact opposite by having odds ratio above 1.

6.3 Boosting (AdaBoostM1 with J48)

Metric	All Variables	Univariate	Multivariate	Wrapper
Accuracy (%)	**69.03**	65.85	66.83	66.44
Cohen's Kappa	**0.4613**	0.4033	0.4238	0.4124
Precision	**0.698**	0.665	0.674	0.672
Recall	**0.690**	0.658	0.668	0.664
F1-score	**0.683**	0.649	0.661	0.655
AUC	**0.797**	0.757	0.772	0.768
RMSE	**0.3844**	0.4041	0.3982	0.3997
Recall (High)	**0.519**	0.483	0.490	0.483
Recall (Medium)	**0.826**	0.805	0.804	0.815
Recall (Low)	**0.557**	0.507	0.541	0.510
High → Low (%)	**1.24**	1.83	2.12	2.27
Medium → Low (%)	**10.67**	13.49	13.45	12.39
Low → High (%)	0.82	1.00	1.00	**0.63**

Table 12: AdaBoostM1 with J48 performance across feature selection configurations.

J48 was selected as the base classifier for boosting because decision trees can capture nonlinear relationships and interactions while remaining interpretable. Boosting may improve its performance by sequentially training classifiers that assign greater weight to observations misclassified in previous iterations. This is particularly relevant to avalanche danger classification, where distinguishing between adjacent danger levels is challenging.

The results obtained using all variables show an overall improvement compared with the original J48 model, supporting the potential advantages of boosting. Accuracy and Cohen's kappa increased, as did recall for all classes, while also slightly reducing the critical High to Low errors at the expense of some Medium to Low errors, as we can see from the following confusion matrices.

	Predicted class		
Actual class	High	Medium	Low
High	1350	1342	44
Medium	500	5707	708
Low	52	1766	1721

Table 13: J48 All Variables

	Predicted class		
Actual class	High	Medium	Low
High	1420	1282	34
Medium	463	5714	738
Low	29	1539	1971

Table 14: Boosting All Variables

Table 15: Comparison of confusion matrices

Feature selection did not improve performance, the same way that it didn't for the original J48 algorithm. Decision trees select relevant predictors through their splits, therefore removing features might eliminate predictors that become informative when combined with others. This is particularly relevant for boosting, which repeatedly adjusts its focus towards misclassified observations and can benefit from complementary information across the full feature set.

Despite the overall improvement over standalone J48, the boosting version still struggles to distinguish between adjacent danger levels, particularly High and Medium.

6.4 Random Forest (RandomForest)

6.4.1 Selection of the Number of Trees and Randomly Selected Attributes

Metric	**100 trees**			**500 trees**		
	K=3	**K=5**	**K=10**	**K=3**	**K=5**	**K=10**
Accuracy (%)	72.72	71.15	68.73	**73.06**	71.30	68.89
Cohen's Kappa	0.5268	0.5024	0.4647	**0.5325**	0.5042	0.4671
Precision	0.736	0.717	0.689	**0.740**	0.719	0.691
Recall	0.727	0.712	0.687	**0.731**	0.713	0.689
F1-score	0.721	0.706	0.682	**0.724**	0.707	0.684
AUC	0.851	0.835	0.813	**0.855**	0.839	0.815
RMSE	0.3581	0.3647	0.3755	**0.3565**	0.3631	0.3744
Recall (High)	0.533	0.527	0.512	**0.538**	0.525	0.511
Recall (Medium)	0.855	0.830	0.794	**0.858**	0.833	0.796
Recall (Low)	0.628	0.624	0.615	**0.631**	0.623	0.618

Table 16: RandomForest performance using different numbers of trees and randomly selected attributes, with all variables.

In order to select the optimal number of trees and number of randomly selected attributes, different values were tested using all variables.

Regarding the number of trees, increasing it from 100 to 500 produces a slight but consistent improvement in the evaluation metrics, particularly when $K = 3$. This suggests that using more trees helps stabilize the predictions, although the improvement is relatively small.

The number of randomly selected attributes also has a clear effect. Increasing the number consistently worsens overall performance, especially for $K = 10$. This suggests that restricting the number of candidate attributes at each split to a smaller subset allows the trees to capture more useful patterns while maintaining diversity.

Taking these patterns into account, the configuration with 500 trees and $K = 3$ was selected for the final RandomForest model and upcoming testing.

6.4.2 Model Interpretation

Metric	All variables	Univariate	Multivariate
Accuracy (%)	**73.06**	67.37	69.72
Cohen's Kappa	**0.5325**	0.4473	0.4864
Precision	**0.740**	0.674	0.699
Recall	**0.731**	0.674	0.697
F1-score	**0.724**	0.670	0.694
AUC	**0.855**	0.796	0.816
RMSE	**0.3565**	0.3870	0.3750
Recall (High)	0.538	0.568	**0.583**
Recall (Medium)	**0.858**	0.764	0.786
Recall (Low)	**0.631**	0.579	0.612
High → Low (%)	**1.50**	3.29	2.27
Medium → Low (%)	**8.88**	15.12	13.54
Low → High (%)	**0.45**	1.84	1.33

Table 17: RandomForest performance with 500 trees and $K = 3$.

The results show that RandomForest achieves its best overall performance with all variables. This suggests that the excluded variables contain complementary information that contributes to the classification. Moreover, RandomForest is able to handle a relatively large number of predictors through the random feature selection mechanism. At each split, only a random subset of attributes is considered, which introduces diversity among the trees and reduces their dependence on individual predictors.

Consequently, retaining all variables allows different trees to exploit different combinations of features, potentially capturing relationships that would be lost in univariate or multivariate filter. For this reason, Wrapper feature selection was not performed.

Interestingly, the Univariate and Multivariate configurations achieve a higher recall for the High class (0.568 and 0.583 vs 0.538), but at the cost of a much higher proportion of High and Medium forecasts classified as Low. Since underestimating the danger is still the main concern, the All Variables configuration remains preferable.

7 Final Model and Conclusions

7.1 Comparison of the best configurations

Table 18 compares the best configuration of each algorithm, as selected in the previous sections.

Metric	k-NN	C4.5	RIPPER	SVM	Logistic	Stacking	Boosting	Random Forest
Variables	All	All	All	All	Wrapper	All	All	All
Accuracy (%)	69.58	66.55	68.31	68.99	69.64	71.24	69.03	**73.06**
Cohen's Kappa	0.496	0.413	0.4485	0.4534	0.4708	0.5024	0.4613	**0.5325**
Precision	0.695	0.673	0.690	0.705	0.706	0.720	0.698	**0.740**
Recall	0.696	0.666	0.683	0.690	0.696	0.712	0.690	**0.731**
F1-score	0.695	0.655	0.675	0.680	0.689	0.707	0.683	**0.724**
AUC	0.801	0.749	0.742	0.757	0.808	0.827	0.797	**0.855**
RMSE	0.384	0.3989	0.3934	0.3909	0.3758	0.3653	0.3844	**0.3565**
Recall (High)	**0.619**	0.493	0.501	0.485	0.524	0.565	0.519	0.538
Recall (Medium)	0.751	0.825	0.819	0.849	0.835	0.836	0.826	**0.858**
Recall (Low)	**0.647**	0.486	0.558	0.537	0.558	0.585	0.557	0.631
High → Low (%)	2.56	1.61	1.72	**1.06**	1.54	1.21	1.24	1.50
Medium → Low (%)	12.75	10.24	11.27	10.54	10.78	9.95	10.67	**8.88**
Low → High (%)	4.15	1.47	0.40	0.51	0.62	**0.37**	0.82	0.45

Table 18: Best configuration of each algorithm ($k = 3$ for k-NN, linear kernel for SVM, 500 trees and $K = 3$ for Random Forest (RF)).

7.2 Strengths and weaknesses

As a reference, a ZeroR classifier that always predicts Medium would reach an accuracy of 52.4% (6915/13190), so all the models improve substantially on this baseline.

- **k-NN:** simple and with the highest recall for the High and Low classes. However, its performance depends on k, which trades High recall for Medium recall, it is not interpretable, and it has the highest Medium → Low rate (12.75%) and the most false alarms (4.15% of Low predicted as High).
- **C4.5:** produces a compact tree whose main splits (rainfall, temperature and time of the season) agree with physical intuition. However, it has the lowest accuracy, it is biased towards the Medium class, and some branches are overfitted.
- **RIPPER:** the 36 rules are easy to read and can be checked against domain knowledge, but its performance is below the best models. Most observations fall under the default Medium rule, and some rules combine many conditions and may be overfitted.
- **Logistic Regression:** provides interpretable odds ratios that quantify the effect of each variable, and with wrapper selection it reaches 69.64% accuracy. However, it gives unstable estimates for rare categories.
- **SVM:** the linear kernel is competitive and has the lowest High → Low rate (1.06%). However, its High recall is the lowest of all models (0.485) and it's computationally expensive.
- **Stacking:** combines complementary base models, which gives balanced recall values and the second best performance in terms of critical misclassifications. However, it is harder to interpret and requires training several models.
- **Boosting:** clearly improves on a single J48 (69.03% versus 66.55% accuracy) and increases the recall of every class. However, it becomes harder to interpret since there are more than a single tree and with different weights assigned.
- **Random Forest:** has the best performance overall in most of the metrics. However, it is less interpretable, its High recall (0.538) is lower than that of k-NN and Stacking, and it is sensitive to the number of attributes K.

7.3 Final Model

Random Forest with all variables (500 trees, $K = 3$) is selected as the final model. It performs best on most of the metrics in Table 18 and it gives the lowest combined number of High and Medium forecasts underestimated as Low, which is the main priority of this project. Its advantage

comes from averaging many trees trained on random subsets of attributes, which reduces the instability and overfitting observed in a single C4.5 tree.

However, it is not the best model in every respect. k-NN detects more High and Low forecasts, SVM, Stacking and Boosting make slightly fewer High → Low errors, and many High forecasts are still predicted as Medium. In addition, an ensemble of 500 trees cannot be read in the way a single tree or a set of rules can.

Stacking is the closest alternative. Its recall for the High class is higher and it produces the fewest false alarms, although with lower overall accuracy and a more complex training process (three base models and a meta-classifier). Its odds ratios also give some insight into how much weight the meta-classifier gives to each base model. If interpretability were more important than predictive performance, **Logistic Regression** would be preferable, since its odds ratios quantify the effect of each variable, while C4.5 and RIPPER offer rules that can be read directly, at the cost of lower accuracy.

7.4 Limitations and Conclusions

This dataset relies heavily on meteorological conditions and it contains no information about the slope or the snowpack (snow condition and stability), which forecasters consider when assigning a danger level. Consequently, this project is mainly focused on predicting expected danger from weather and temporal variables alone. In addition, the forecasts are produced by professionals, so there is a human component in the final decision that a machine learning algorithm cannot fully capture.

Despite these limitations, the final Random Forest model considerably reduced the High and Medium forecasts misclassified as Low, which in a real-life situation are the errors that matter most. At the same time, the algorithms consistently identified rainfall (especially in the most exposed area), wind speed and wind changes, maximum temperature and time of the season as the conditions most related to the danger level. These results are consistent with the physical processes behind avalanche formation, such as wet snow caused by rain and warm temperatures, and snow loading caused by wind. However, the confusion between High and Medium, and Medium and Low remains, so these models should be seen as a support tool and not as a replacement for the forecasts of the experts.

References

[1] European Avalanche Warning Services (EAWS). Avalanche problems. https://www.avalanches.org/standards/avalanche-problems/, .

[2] European Avalanche Warning Services (EAWS). European avalanche danger scale. https://www.avalanches.org/standards/avalanche-danger-scale/, .

[3] Norwegian Water Resources and Energy Directorate (NVE). Avalanche warning api. https://api.nve.no/doc/snoeskredvarsel/.

Annex A: Feature selection results

Table A.1 shows the variables selected by each feature selection method, ordered by average Information Gain.

					Wrapper			
Variable	**Avg. IG**	**Univ.**	**Folds**	**Multiv.**	**k-NN**	**C4.5**	**RIPPER**	**Logistic**
rainfall_most_exposed_area	0.199	X	10/10	X	X	X	X	X
rainfall_average	0.191	X	10/10	X				X
wind_shift_speed	0.145	X	10/10	X		X	X	X
wind_speed	0.141	X	10/10	X		X		X
publish_month	0.136	X	10/10	X	X	X	X	X
temperature_max	0.132	X	10/10	X			X	X
temperature_min	0.084	X	0/10			X		X
cloud_cover	0.082	X	10/10	X	X			
freezing_level_masl	0.066		5/10	X				
RegionName	0.063		0/10					X
rain_snow_line_altitude_masl	0.055		0/10		X		X	
publish_hour	0.049		10/10	X	X		X	X
wind_shift_end	0.035		2/10					
wind_shift_start	0.034		0/10					X
wind_direction	0.028		0/10					
geographical_region	0.028		0/10		X	X		
wind_shift_direction	0.022		0/10				X	
wind_direction_change	0.019		0/10					
rain_snow_line_end	0.018		0/10		X	X		X
rain_snow_line_start	0.012		0/10		X			
freezing_level_end	0.005		0/10					X
freezing_level_start	0.004		0/10		X			X
Total selected		8		9	9	7	7	13

Table A.1: Variables selected by each feature selection method.

The univariate filter keeps the attributes whose average Information Gain is above the global average 0.07036. A univariate filter using Gain Ratio was also used and it resulted in the same selection of variables.

For the multivariate filter, the BestFirst approach was used. The Folds column indicates the number of folds in which each attribute was selected, providing an estimation of selection stability. Attributes selected in 5 or more folds have been included in the multivariate filter.

The univariate and multivariate filter feature subset selection have been done using 10-fold cross-validation. The filters differ only in two variables so it is expected that the evaluation results will be similar for both of them.

The Wrapper feature selection was run with a greedy approach. Wrapper for Support Vector Machine was too computationally expensive and for metaclassifiers was potentially redundant.